

SECUREVAULT

SDLC LIFECYCLE MAPPING REPORT



SUBMITTED BY:

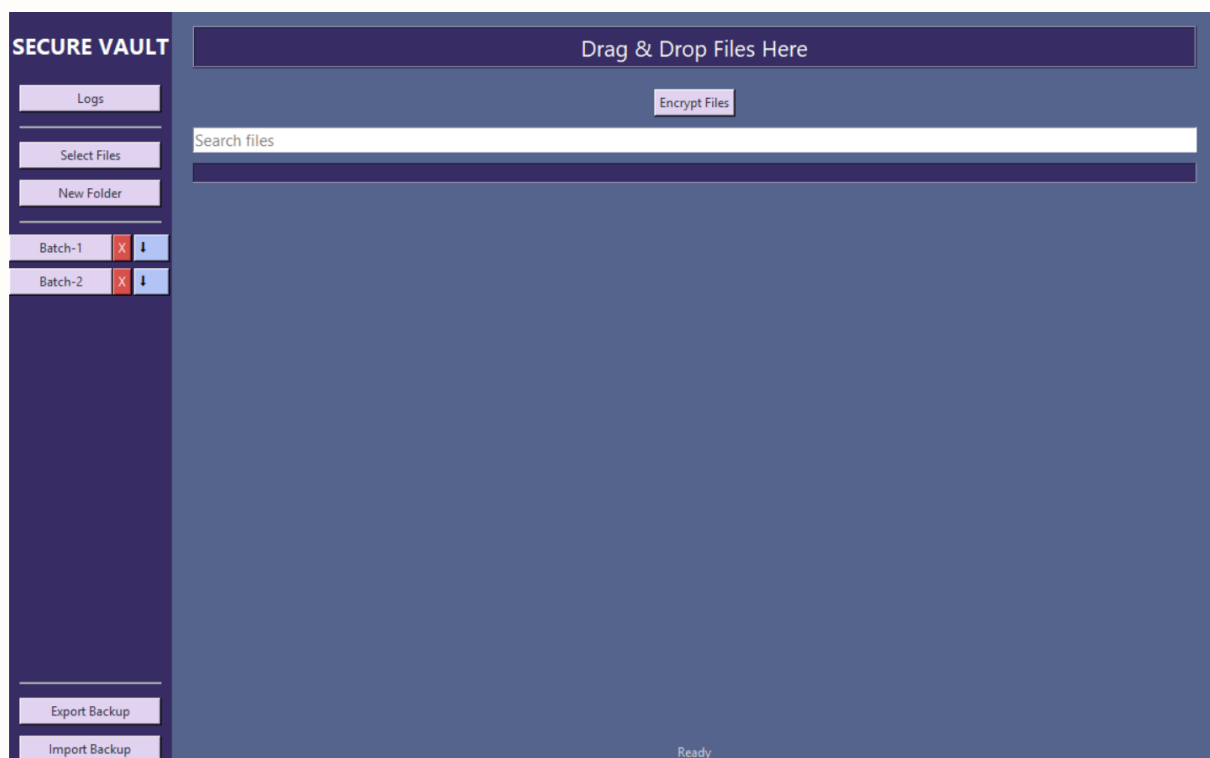
Team Cipher Syndicate

INTRODUCTION

SecureVault is a desktop-based encrypted file management system developed to securely store, encrypt, organize, and protect sensitive digital files.

The application combines a Python-based graphical user interface with a C-powered encryption backend using OpenSSL cryptography libraries.

The project was designed to address growing concerns related to unauthorized file access, insecure local storage, accidental data leaks, and weak file protection systems.



REQUIREMENT ANALYSIS

Problem Statement:

Sensitive digital files are frequently exposed to risks such as unauthorized access, accidental leaks, insecure deletion, and brute-force attacks. Many users lack simple and secure desktop tools capable of protecting confidential files locally while maintaining usability and performance.

Target End Users:

SecureVault was designed for:

- ❖ Students
- ❖ Professionals
- ❖ Small businesses
- ❖ Privacy-conscious users
- ❖ Users storing confidential local files

The goal was to create a *secure yet user-friendly* desktop vault system capable of handling encrypted storage, secure deletion, and intrusion monitoring.

SYSTEM DESIGN

The system architecture was divided into two major components:

Python-Based GUI

Responsible for:

- User interface
- File management
- Drag-and-drop support
- Search filtering
- Logging systems
- Progress tracking

C Encryption Backend

Responsible for:

- AES-256-GCM encryption
- AES-256-GCM decryption
- PBKDF2 password key derivation
- File authentication and integrity verification

The backend communicates with the Python GUI through subprocess execution.

DEVELOPMENT PROCESS

The project was developed using:

- Python
- Tkinter
- C Programming
- OpenSSL Cryptography Library

The key features implemented during development included:

- ★ Encrypted vault folders
 - ★ Password-protected access
 - ★ Drag-and-drop uploads
 - ★ Live file search
 - ★ Intrusion detection logging
 - ★ Secure file shredding
 - ★ Encrypted backup export and recovery
 - ★ Real-time encryption progress tracking
-

TESTING AND RESILIENCE

SecureVault was tested against multiple edge cases and unexpected user interactions to improve system stability and security. The following are some examples:

Validation & Error Handling Implemented

- ★ Empty file selection prevention
- ★ Incorrect password detection
- ★ Temporary vault lockout after repeated failed attempts
- ★ Missing backend executable handling
- ★ Secure deletion failure handling
- ★ Corrupted output cleanup after failed decryption
- ★ Tampered file detection using AES-GCM authentication tags
- ★ Thread-safe GUI updates during encryption

The application uses message-based feedback systems to safely notify users instead of crashing during invalid operations.

REFACTORING AND OPTIMIZATION

During development, the project codebase became increasingly difficult to maintain due to a large monolithic application structure.

Before Optimization

- Single 1000+ line app.py file
- Tightly coupled functionality
- Difficult feature scaling and debugging
- GUI freezing during large encryption operations
- Manual dependency setup required to run this application
- Slower cryptographic processing workflow

After Optimization

The application was refactored into modular reusable components. Additional optimizations included:

- ★ Background threaded encryption to prevent GUI freezing
- ★ Real-time progress bar synchronization
- ★ Dynamic live file filtering
- ★ Organized color-coded logging systems
- ★ Improved scalability and maintainability
- ★ Standalone executable packaging for improved portability across systems

These improvements significantly increased maintainability, scalability, responsiveness, and code readability.

SECURITY IMPROVEMENTS

Several security-focused improvements were implemented throughout the project lifecycle.

Cryptographic Security

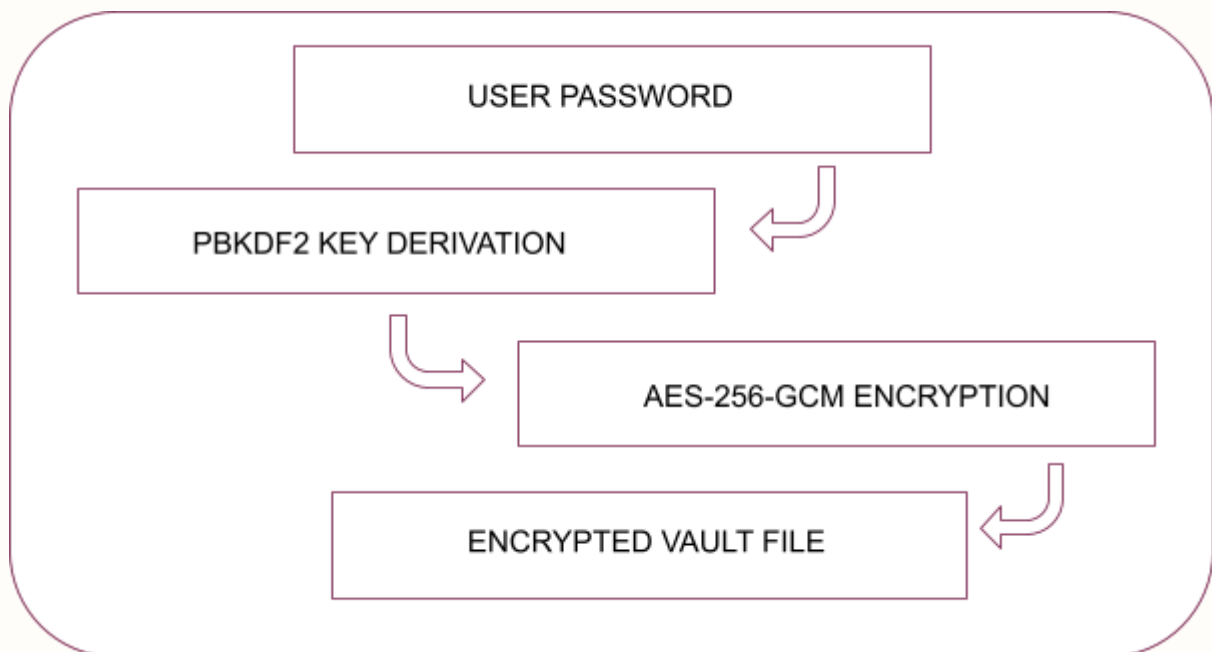
- AES-256-GCM authenticated encryption

- PBKDF2-HMAC-SHA256 password key derivation
- Randomized salts
- Randomized initialization vectors (IVs)

System Security

- Intrusion detection logs
- Temporary vault lockouts
- Secure file shredding
- Password strength analysis
- Encrypted backup portability

These additions improved resistance against unauthorized access, brute-force attacks, encrypted data leaks, and forensic file recovery.



CONCLUSION

SecureVault evolved through continuous testing, refactoring, and optimization, the project demonstrates practical implementation of secure software engineering principles while maintaining an accessible and responsive user experience.